Kafka 的真實案例

它在今天的資訊世界扮演什麼角色

LinkedIn ・ Uber ・ LINE ・ Netflix ・ AO.com ・ BRI ・ The New York Times ・ Michelin

課程目標

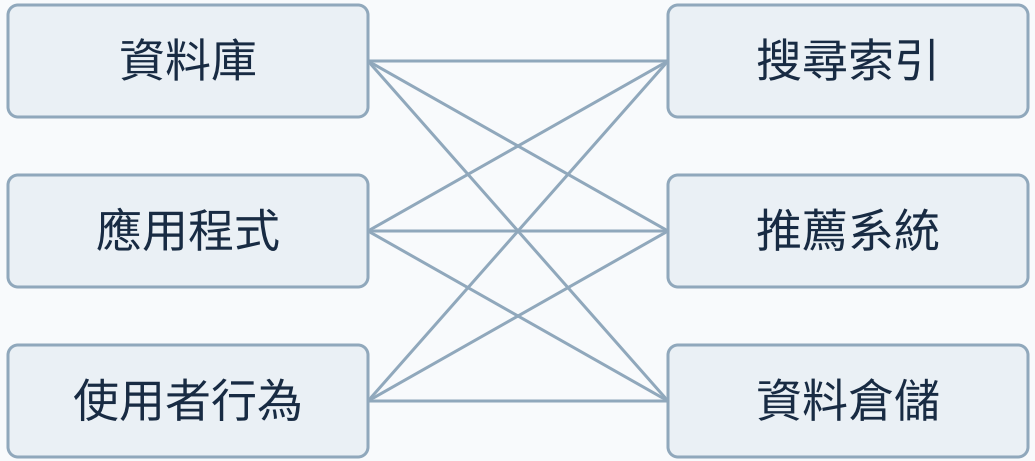
1. 說出 Kafka 為什麼誕生，以及它想解決的問題
2. 分辨 Kafka 在系統裡常見的**四種角色**，並各舉一個公司案例
3. 理解不同公司為什麼做出不同選擇
    - 要平行度，還是要順序？
    - 自己維運，還是交給雲端？
4. 讀到新的案例時，能判斷 Kafka 的角色與代價

一、從 LinkedIn 開始

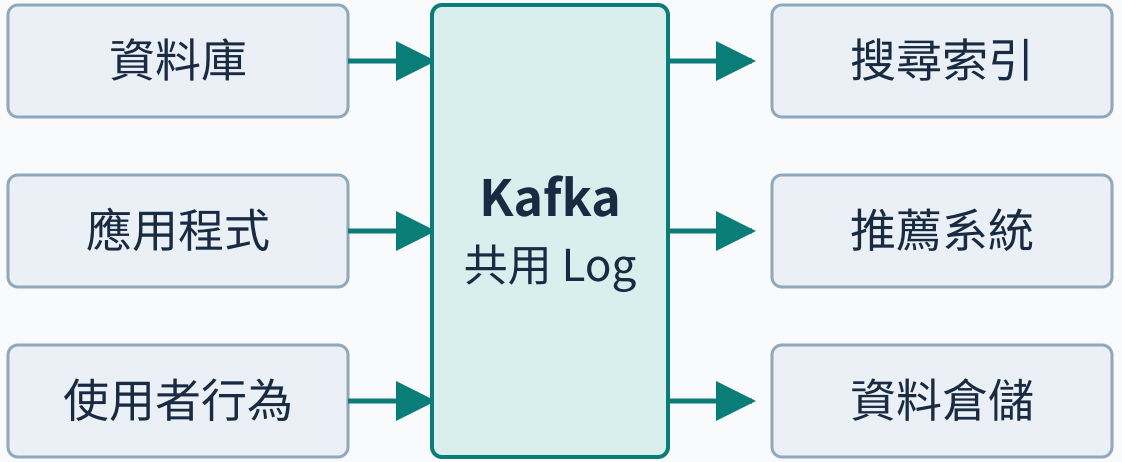
Kafka 為什麼誕生？

問題：每個系統都要互相串接

直接串接：3 × 3 = 9 條



透過 Kafka：3 + 3 = 6 條



中間放一條共用的 Log，串接數從 N × M 降為 N + M

LinkedIn 的成長：八年七千倍



每天處理的訊息量（線性刻度：前三年的量小到幾乎看不見）

2019 年的 LinkedIn

100+	4,000+	10 萬+	700 萬
叢集	Broker	Topic	Partition

- Kafka 開源後，三位作者離開 LinkedIn，創立了 Confluent
- LinkedIn 跑自己維護的 `-li` release branch，開源在 GitHub
  - 強調不是 fork，盡量貼近 upstream，把修改回饋給 Apache Kafka
- **開源循環**：公司在生產環境遇到問題，修好後貢獻回社群

Kafka 的四種角色

角色	Kafka 在做什麼	案例
服務之間的事件骨幹	服務發布事件，其他服務各自訂閱	Uber、LINE
資料收集與整合管線	把資料持續搬到需要的地方	Netflix、AO.com
即時串流處理	資料流動的同時就完成計算	BRI
Source of truth	Log 本身就是正本	The New York Times

真實系統常同時扮演好幾種角色，分類只是幫助你讀案例時抓住重點

三、角色一：服務之間的事件骨幹

當服務從 5 個變成 1,000 個，會發生什麼事？

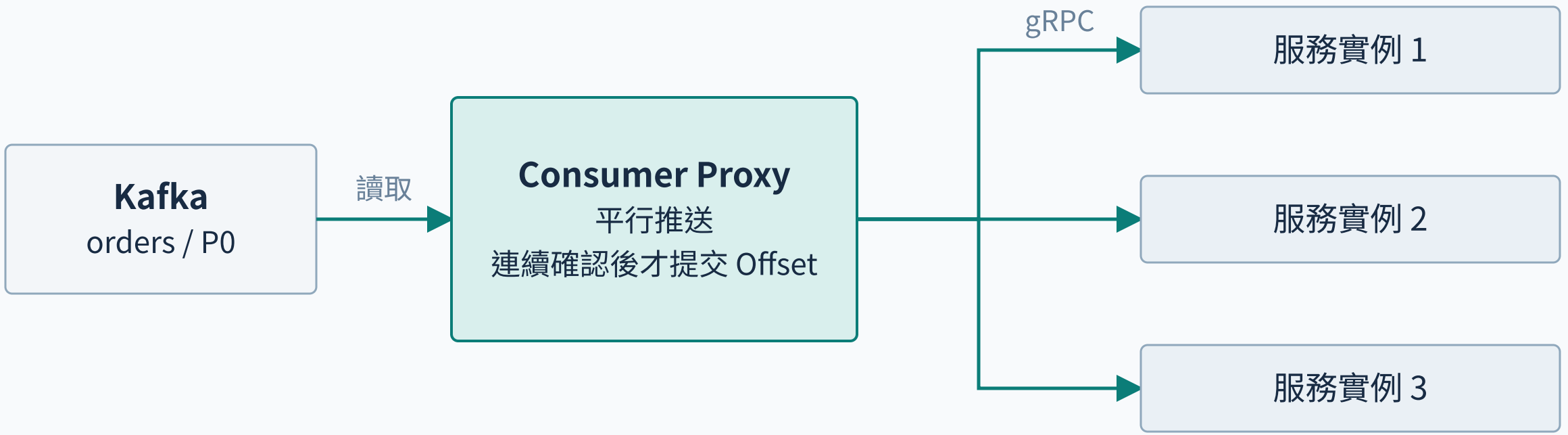
Uber：上千個微服務的非同步佇列

數兆則	300+	100 萬 → 1,200 萬
每天訊息量	微服務用 Kafka 做佇列	五年內每秒訊息量

直接用 Kafka Client 遇到三個問題：

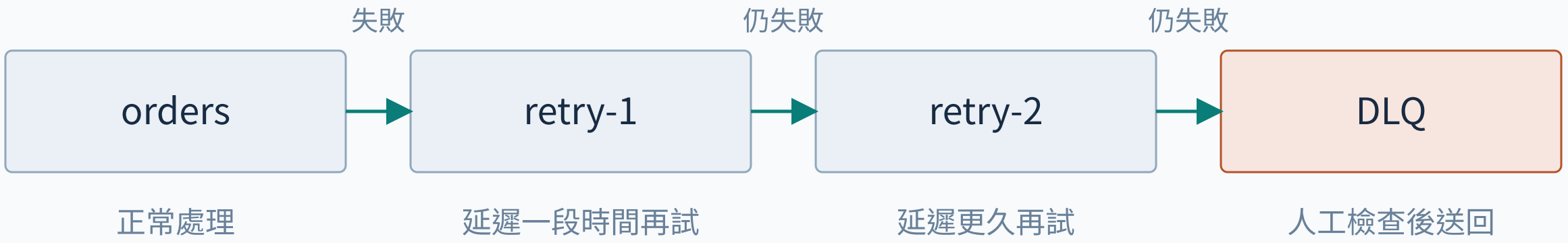
1. **語言太多**：Go、Java、Python、Node.js，升級 1,000 多個服務要好幾個月
2. **平行度被 Partition 數限制**：處理一筆要 1 秒，每秒 1,000 筆就要 1,000 個 Partition
3. **一筆卡住，後面全卡住**：慢訊息或 poison pill 擋住整個 Partition

Uber 的解法：Consumer Proxy



服務只要提供 gRPC endpoint，不必內建 Kafka Client；平行度不再受 Partition 數限制

Uber：Retry Topic 與 Dead Letter Queue



原本的 Consumer 轉寫到 retry Topic 後就提交 Offset，繼續處理下一筆

失敗的訊息不能卡住整條資料流

LINE（LY Corporation）：你每天都在用的服務

1 兆+	2.6 PB	3,100 萬	200+
每天訊息量	每天 I/O	尖峰每秒訊息	Broker

- 用途：pub-sub、日誌管線、非同步處理、遊戲資料、CDC 同步資料庫
- 「只要服務會產生資料，多半就會經過 Kafka」

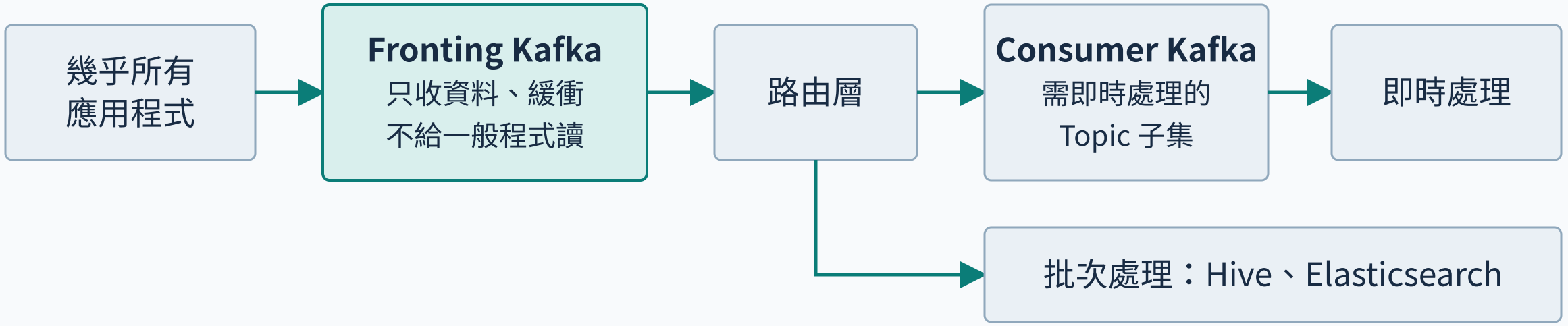
公開資料**沒有**說聊天訊息本身透過 Kafka 傳送

分清楚「公司用 Kafka」和「這個功能用 Kafka」

四、角色二：資料收集與整合管線

把資料從產生的地方，持續搬到需要的地方

Netflix：Keystone 資料管線



36 個叢集、4,000+ Broker、每天 7,000 億則訊息（2016）

Netflix 接受極少量遺失

當時每天的資料遺失率**低於 0.01%**

資料	遺失一點點可以嗎？
使用者行為紀錄	可以，統計結果幾乎不受影響
付款、訂單	不行

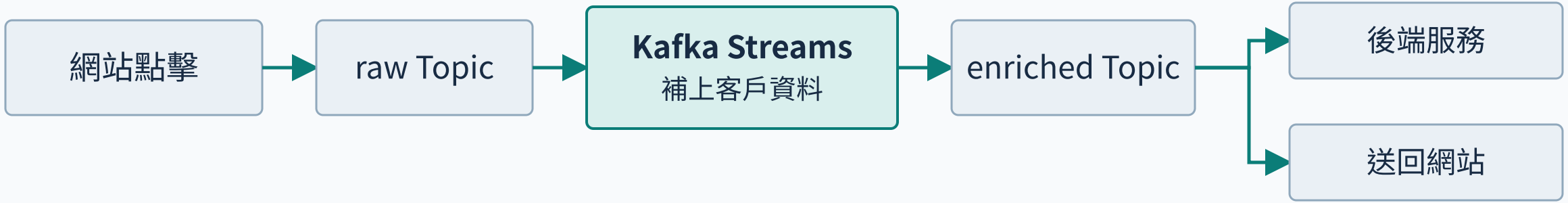
introduction.md 第六章的 `acks` 、Replication、`min.insync.replicas` ，在真實世界就是依資料的重要性來選擇

AO.com：從 CDC 到即時個人化

第一階段：CDC



第二階段：點擊流個人化



AO.com 的兩個重點

保留 raw Topic

enrichment 邏輯有錯，修好後從 raw Topic 重跑：這就是 **Replay**

資料變成事件流之後，其他團隊就會加入

AO 提到又有兩三個團隊開始使用這些資料，這就是第一章 **N + M** 的實際效果

五、角色三：即時串流處理

在資料流動的同時就完成計算

BRI（印尼人民銀行）：即時詐欺偵測

印尼最大的銀行，約 **7,500 萬**名客戶

- 即時信用評分
- 詐欺偵測：標記異常交易、凍結被盜用的卡片
- 商家評估
- 預警系統：在違約發生前，找出可能還不出錢的客戶

詐欺偵測用 KSQL 搭配 Kafka Streams

過去	→	現在
兩週到一個月		**即時**

串流處理在算什麼

- **無狀態**：一次只看一筆，例如「金額超過 10 萬就標記」
- **有狀態**：需要記住過去，例如「同一張卡 1 分鐘內刷超過 5 次」

```
builder.stream("transactions", Consumed.with(Serdes.String(), Serdes.String())) // Key = 卡號
       .groupByKey()
       .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1)))
       .count()
       .toStream()
       .filter((windowedCard, count) -> count > 5)
       .map((windowedCard, count) -> KeyValue.pair(windowedCard.key(), count + " 筆交易"))
       .to("fraud-alerts", Produced.with(Serdes.String(), Serdes.String()));
```

用卡號當 **Key**，同一張卡的交易才會進同一個 Partition

讀客戶案例要注意的事

BRI 的案例是 Confluent 發布的客戶故事

- 內容大多引述 BRI 主管的說法
- 數字是**自行回報**，沒有經過第三方驗證

分清楚兩件事：

技術架構的描述	行銷性質的成效數字
用了哪些元件、資料怎麼流	「從一個月變成即時」「成本省 35%」
通常比較可信	要保留判斷

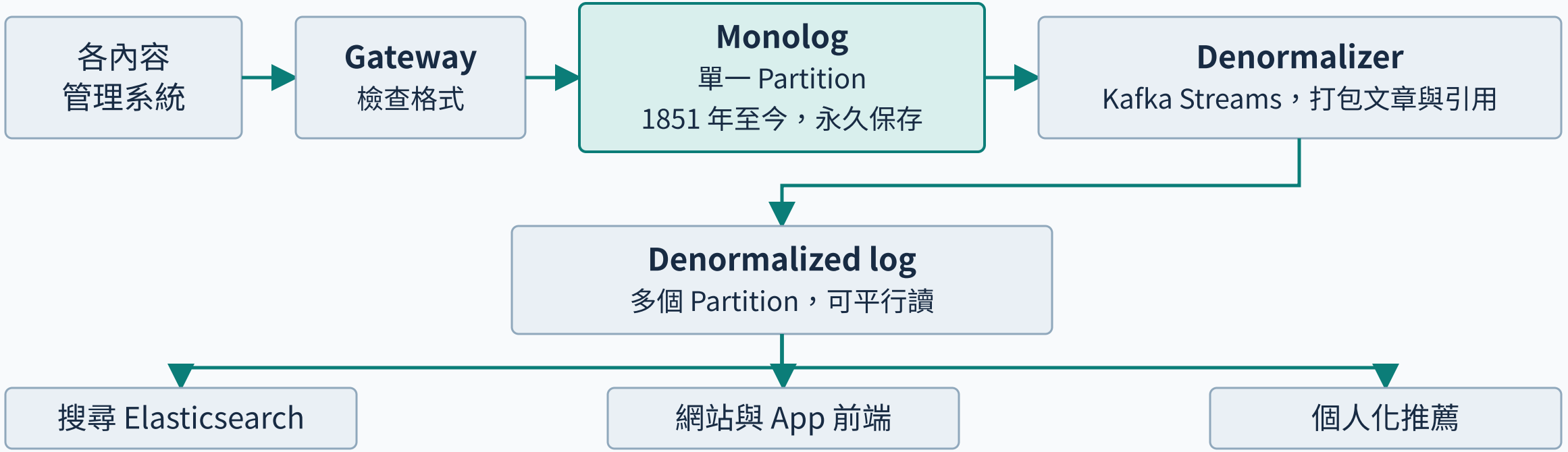
六、角色四：Source of truth

Kafka 本身就是正本，其他系統是衍生出來的

紐約時報原本的問題

- 好幾套內容管理系統，加上 **161 年**的報導存檔
- 搜尋、個人化推薦、網站與 App 都需要讀取已發布的內容
- 過去每套系統各自提供 API
  - 不同團隊在不同時期開發，欄位名稱與格式都不一致
  - 想一次讀取全部歷史內容非常困難

紐約時報的 Publishing Pipeline



為什麼可行

- 新系統上線：從 **Offset 0** 開始讀，就能建出完整的歷史資料
- 搜尋索引要改格式：建一個新的，從頭 **Replay**，追上後再切換
- 不選 Pub/Sub、SQS、Kinesis：需要**永久保存**與**有序讀取**，只有 Kafka 都做到

需要順序的地方犧牲平行度，不需要的地方再把平行度拿回來

七、自己維運，還是交給雲端？

兩類公司

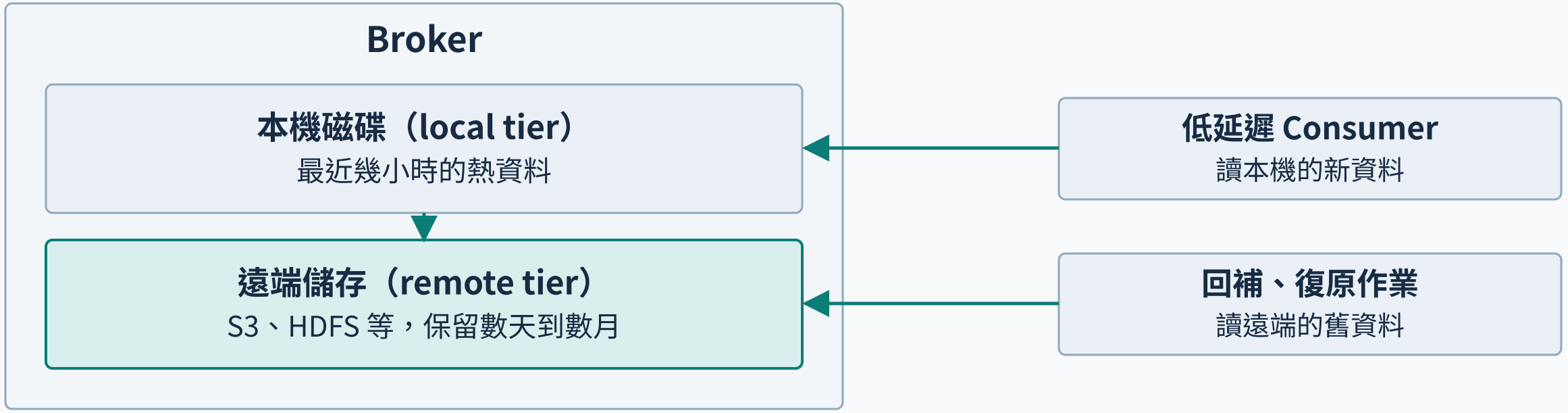
	LinkedIn、Uber、LINE、Netflix	Michelin、AO.com
怎麼運作 Kafka	自己維運大型叢集	從自己維運改用雲端託管
原因	規模極大，基礎設施是核心能力	想把人力放在業務上
和社群的關係	常把改進貢獻回 Apache Kafka	主要是使用者

Michelin：從批次報表到即時庫存

- 庫存報表不可靠、資料過時，影響全球供應鏈
- 舊系統「非常批次、非常 monolithic」
- 先在機房自架 Kafka：需要**三名全職員工**維護，仍難以擴展
- 2021 年改用雲端託管，估計營運成本省下 **35%**
- 用 Kafka Streams 改寫流程編排系統，把 monolith 拆成可各自演進的服務

AO.com 也有類似經驗：自己維運時，一次 Broker 故障最多要**三天**才能解決

大公司把改進貢獻回社群：Tiered Storage



Uber 提出 **KIP-405**，和社群一起完成，3.6.0 以 early access 釋出

Kafka 今天的位置

- **通訊協定成為業界標準介面**：許多雲端廠商與公司提供相容的服務或實作
- **和其他系統結合**：搭配 Flink 做串流計算；Topic 轉成 Iceberg 表格供分析
- **AI 需要即時資料**：詐欺偵測、推薦系統、AI agent 都需要知道「現在正在發生什麼」

Kafka 做的事和 2010 年一樣：讓需要資料的系統，即時拿到來源系統產生的事件

八、怎麼讀一篇 Kafka 案例

1. **原本的痛點是什麼？** 批次太慢、串接太多、服務耦合、還是維運成本？
2. **Kafka 扮演哪一種角色？** 事件骨幹、整合管線、串流處理、source of truth？
3. **付出了什麼代價？** 紐約時報放棄平行度、Netflix 接受少量遺失、Uber 多一層 Proxy
4. **這篇文章是誰寫的？** 工程部落格偏技術細節，客戶案例偏成效

這四個問題也可以直接用在作業的「**專案在解什麼**」

討論題

1. Uber 讓平行度不受 Partition 數限制，紐約時報卻刻意只用一個 Partition。需求差在哪？各自放棄了什麼？
2. 舉一種可以接受遺失的資料，以及一種絕對不行的資料。Kafka 的設定應該有什麼不同？
3. 學校選課系統在開放第一分鐘湧入大量請求。用 Kafka 重新設計，你會建立哪些 Topic？Key 選什麼？

討論題（續）

4. 紐約時報永久保存所有內容。如果用同樣方式保存使用者個資，會遇到什麼問題？（提示：刪除權）
5. 挑一個你常用的服務（外送、叫車、社群、串流影音），推測它背後有哪些事件流，各是哪一種角色？
6. 為什麼 Michelin 選擇託管服務，而 Uber 選擇自己維運，還把 Tiered Storage 貢獻回社群？

延伸閱讀與資料來源

- 完整講義與所有出處：`campus-courses/kafka/real-world-cases.md`
- Jay Kreps，*The Log: What every software engineer should know about real-time data's unifying abstraction*
- Martin Kleppmann，*Turning the database inside-out*、*Designing Data-Intensive Applications*
- KIP-405: Kafka Tiered Storage

案例來源：LinkedIn、Uber、Netflix 工程部落格；ClickHouse 部落格（LY Corporation）；Confluent 部落格（The New York Times）與客戶案例（AO.com、BRI、Michelin）